

3 a.m. Blast Radius

Hands-on lab · about 45 minutes · Neo4j Browser

03:07. Your pager goes off: `profile-cache p99 latency 4.2s, hit rate 31% and falling`. It's a cache. Declared tier 3. It can probably wait until morning.
Can it?

You're the on-call engineer at a large retail bank. Over the next 45 minutes you'll use a graph of the bank's systems to work out **what is actually broken, for whom, how sure anyone can be, and who to wake up**. You will also change the graph yourself and watch the answer move.

By the end you'll have:

- turned **19 alerts from 11 teams** into one likely cause;
- measured a **blast radius**, and separated what is *proven* from what is only *assumed*;
- found the **one link** that nobody ever checked, and seen how it got there;
- **changed the graph** and watched the answer change;
- decided what to do before the morning rush.

Everything happens in your web browser. **There is nothing to install.**

Everything here is fictional. The bank, its teams, services, incidents and customer numbers are invented for this exercise. Nothing describes any real institution's architecture.

Printed copy. Queries copy out of this PDF correctly in macOS Preview and in Chrome. If a pasted query ever fails, copy it from `LAB-GUIDE.md` or the files in `queries/` in the repository or zip instead.

Before the session: prerequisites

What you need

A laptop	Your corporate laptop is fine. Bring it charged
A current web browser	Chrome or Edge recommended
Your lab credentials	From the pre-event email: username, password and the Neo4j Browser URL
Two minutes, at least a day before	For the readiness check below. Do it from the laptop and network you'll bring
Neo4j or Cypher experience	Not required. Every query is provided: you copy, paste and read. If you know Cypher, there are stretch challenges at the end

You get your own database. Nobody else can see it or change it, and you can't break anyone else's. **This lab uses Neo4j Browser only.**

The readiness check (do this before the session)

1. Open the **Neo4j Browser URL** from your pre-event email and log in.
2. Paste this into the editor at the top of the page and press **Ctrl+Enter** (Windows) or **Cmd+Enter** (Mac):

```
CALL db.info() YIELD name
CREATE (:Checkin {at: datetime()})
RETURN name AS your_database,
       COUNT { (:Component) } AS components,
       COUNT { (:Incident) } AS incidents,
       'You are ready' AS status;
```

3. You should see one row like this, with **your own database name** where this example says `lab-user01`:

your_database	components	incidents	status
"lab-user01"	232	228	"You are ready"

1 row

If **components** shows 0, you get an error saying something "is not allowed" on database `neo4j`, or you can't log in: don't try to fix it yourself. Email the contact in your pre-event email with a screenshot. It almost always means your login opened the wrong database, and that is a one-minute fix on our side. The check also leaves a small marker in your database, which is how we know who is ready.

How the lab works

- The presenter runs every step on the big screen at the same time as you. If something doesn't work on your laptop, keep watching; you won't miss the point.
- Run one query at a time. Paste it, run it, read the result, then clear the editor for the next one.
- Copy the whole block, from the first line to the semicolon at the end.
- Queries stand alone, so if you fall behind, skip ahead to where the room is. The one exception is Part 4, which changes the graph: do its steps in order.
- Predict first. Where you see **Predict**, say your guess out loud or to your neighbour before you run the query. It's more fun and you'll remember more.
- Stuck? Raise your hand. Helpers are walking the room.

Neo4j Browser in 60 seconds

- The editor is the box at the top. Paste a query there and press **Ctrl+Enter** / **Cmd+Enter**, or click the ► button.
- Each result appears in a frame below, newest on top. These queries return rows, which Browser shows as a table.
- To clear old frames, type `:clear` and run it.

Reading Cypher in 60 seconds

Cypher describes patterns the way you would draw them on a whiteboard:

```
MATCH (s:Service)-[:DEPENDS_ON]->(d:Datastore)
```

This reads: a *Service* `s` that *DEPENDS_ON* a *Datastore* `d`. Round brackets are **things**, square brackets are **connections**, and `:Word` is a label or a connection type. `WHERE` filters, and `RETURN` says what to show. `-[:DEPENDS_ON*1..8]->` means "follow `DEPENDS_ON` connections, between 1 and 8 in a row".

That is enough to read every query in this lab.

The world you're investigating

A bank's technology estate, as a graph:

```
(:Team) -[:OWNS]-> (:Component) <-[:ROOT_CAUSE]- (:Incident) -[:ASSIGNED_TO]-> (:Team)
      |
      |[:DEPENDS_ON]
      v
      |
      |[:DEPLOYED_TO]- (:Change)
      |
      |
      v
  (:Component)      (:Journey) -[:REQUIRES]-> (:Service)
                   (:Alert)   -[:FIRED_ON]-> (:Component)
```

- A **Component** is either a **Service** (code that runs) or a **Datastore** (a database, cache or queue).
- A **Journey** is a *thing customers do*: Log in, Pay a bill, Send a wire. There are 30. Each needs several services to work.
- Every Component has a **tier**: 1 means customer-facing and critical, 3 means it can wait until morning. For a Datastore this is the tier it was **declared** to have.
- **Customer numbers count attempts**. Someone who logs in and then checks a balance is two attempts, so these are not unique people.

Three kinds of dependency

Every `DEPENDS_ON` connection says how much one component needs another:

	meaning
Hard (<code>critical = true</code>)	The caller fails without it
Soft (<code>critical = false</code>)	The caller slows down or falls back
Unclassified (<code>critical</code> is empty)	Nobody ever decided. Common in real systems

Two flags, worked out from that, appear in the queries:

flag	means
<code>confirmed</code>	Known to be hard. This is what you can prove
<code>hard</code>	Not known to be soft, so assume the worst until someone proves otherwise

So *confirmed* is the best case you can defend, and *hard* is the worst case you must plan for. The distance between them is what this lab is about.

Part 1: The page (about 8 minutes)

03:07. It's a tier-3 cache. Surely it can wait.

Step 1: What fired?

```
MATCH (al:Alert {id: 'ALR-77120'})-[:FIRED_ON]->(d:Datastore)<-[:OWNS]-(t:Team)
RETURN al.fired_at AS fired, al.summary AS summary,
       d.name AS component, d.kind AS kind, d.tier AS declared_tier,
       t.name AS owner, t.pager AS pager;
```

You should see one row: a Redis cache called `profile-cache`, **declared tier 3**, owned by Data Platform.

fired	summary	component	kind	declared_tier	owner	pager
2026-09-30T03:07Z	"profile-cache p99 latency 4.2s, hit rate 31% and falling"	"profile-cache"	"redis"	3	"Data Platform"	"dataplat-oncall"

1 row

Step 2: It's not one alert

By 03:14, **19 alerts** have fired from **11 teams** in 7 minutes. Everyone assumes their own service is the problem. The query counts, for each alerting component, how many *other* alerting components depend on it, and how many it depends on.

Predict: how many of the 19 are actually the cause?

```
MATCH (al:Alert)-[:FIRED_ON]->(ac:Component)
WITH collect(DISTINCT ac) AS alerted
MATCH (al:Alert)-[:FIRED_ON]->(c:Component)<-[:OWNS]-(t:Team)
WITH al, c, t,
     size([o IN alerted WHERE o <> c AND EXISTS {
         MATCH p = (o)-[:DEPENDS_ON*1..8]->(c)
         WHERE all(r IN relationships(p) WHERE r.hard)
     }]) AS alerting_dependents,
     size([o IN alerted WHERE o <> c AND EXISTS {
         MATCH p = (c)-[:DEPENDS_ON*1..8]->(o)
         WHERE all(r IN relationships(p) WHERE r.hard)
     }]) AS alerting_dependencies
RETURN al.id AS alert, substring(toString(al.fired_at), 11, 5) AS at,
       c.name AS component, t.name AS team, alerting_dependents, alerting_dependencies,
       CASE WHEN alerting_dependencies = 0 AND alerting_dependents > 0 THEN 'likely cause'
            WHEN alerting_dependencies > 0 THEN 'symptom'
            ELSE 'unrelated' END AS role
ORDER BY alerting_dependents DESC, at, alert;
```

You should see 1 likely cause (it has 9 alerting dependents and depends on nothing that's alerting), 10 symptoms, and 8 unrelated alerts that have no link to any other alerting component in either direction.

alert	at	component	team	alerting_dependents	alerting_dependencies	role
"ALR-77120"	"03:07"	"profile-cache"	"Data Platform"	9	0	"likely cause"
"ALR-77121"	"03:08"	"customer-profile-svc"	"Data Platform"	8	1	"symptom"
"ALR-77123"	"03:09"	"entitlements-svc"	"Identity & Access"	4	2	"symptom"
"ALR-77124"	"03:09"	"auth-gateway"	"Identity & Access"	1	3	"symptom"
"ALR-77127"	"03:11"	"sanctions-screening"	"Payments Platform"	1	2	"symptom"
"ALR-77122"	"03:08"	"image-store"	"Digital Channels"	0	0	"unrelated"
"ALR-77125"	"03:10"	"wire-svc"	"Payments Platform"	0	4	"symptom"
"ALR-77126"	"03:10"	"wire-svc"	"Payments Platform"	0	4	"symptom"
"ALR-77128"	"03:11"	"logs-index"	"Observability"	0	0	"unrelated"
"ALR-77129"	"03:11"	"settlement-db"	"Merchant Services"	0	0	"unrelated"
"ALR-77130"	"03:12"	"bill-pay-svc"	"Payments Platform"	0	3	"symptom"
"ALR-77131"	"03:12"	"email-gateway"	"Messaging Platform"	0	0	"unrelated"
"ALR-77132"	"03:12"	"campaign-svc"	"Marketing Tech"	0	2	"symptom"
"ALR-77133"	"03:13"	"mobile-bff"	"Digital Channels"	0	4	"symptom"
"ALR-77134"	"03:13"	"atm-journal-db"	"Branch & ATM Tech"	0	0	"unrelated"
"ALR-77135"	"03:13"	"trade-finance-svc"	"Corporate Banking"	0	0	"unrelated"
"ALR-77136"	"03:14"	"agent-desktop-svc"	"Customer Support Tech"	0	2	"symptom"
"ALR-77137"	"03:14"	"push-gateway"	"Messaging Platform"	0	0	"unrelated"
"ALR-77138"	"03:14"	"metrics-tsdb"	"Observability"	0	0	"unrelated"

19 rows

What to notice: the cause is the quietest, lowest-tier component in the room. Among the symptoms are two nobody would connect to a cache: campaign-svc and agent-desktop-svc. And 8 of the 19 alerts are noise: image-store and email-gateway have nothing to do with this.

Part 2: How bad is it? (about 12 minutes)

03:14. Now follow the dependencies upstream.

Step 3: The blast radius, in rings

Which services fail if `profile-cache` goes, and how many hops from the cache are they? This is the **worst case**: it follows every `hard` dependency.

```
MATCH p = (s:Service)-[:DEPENDS_ON*1..8]->(:Datastore {name: 'profile-cache'})
WHERE all(r IN relationships(p) WHERE r.hard)
WITH s, min(length(p)) AS hops
ORDER BY coalesce(s.tier, 9), s.name
RETURN hops, count(*) AS services, collect(s.name)[0..5] AS most_critical_examples
ORDER BY hops;
```

You should see 4 rings of 1, 12, 27, 8 services, **48 in all**. (The examples are the most critical in each ring.)

hops	services	most_critical_examples
1	1	["customer-profile-svc"]
2	12	["entitlements-svc", "sanctions-screening", "agent-desktop-svc", "loan-origination-svc", "payee-verification-svc"]
3	27	["auth-gateway", "p2p-transfer-svc", "rtp-svc", "wire-svc", "ach-origination-svc"]
4	8	["mobile-bff", "web-bff", "auth-proxy", "open-account-svc", "order-routing-svc"]

4 rows

Step 4: How much can we prove?

The same radius, split by what you can prove.

Predict: of those 48, how many can you *prove* will fail?

```
MATCH p = (s:Service)-[:DEPENDS_ON*1..8]->(:Datastore {name: 'profile-cache'})
WHERE all(r IN relationships(p) WHERE r.active)
WITH s, collect(p) AS paths
WITH s,
    any(p IN paths WHERE all(r IN relationships(p) WHERE r.confirmed)) AS confirmed,
    any(p IN paths WHERE all(r IN relationships(p) WHERE r.hard)) AS assumed
WITH s, CASE WHEN confirmed THEN 'fails (confirmed)'
             WHEN assumed THEN 'fails (assumed)'
             ELSE 'degrades only' END AS impact
ORDER BY coalesce(s.tier, 9), s.name
RETURN impact, count(*) AS services, collect(s.name)[0..6] AS most_critical_examples
ORDER BY CASE impact WHEN 'fails (confirmed)' THEN 1 WHEN 'fails (assumed)' THEN 2 ELSE 3 END;
```

You should see three groups: **14** proven, **34** assumed (only an unclassified link keeps them on the list), and **28** that merely degrade.

impact	services	most_critical_examples
"fails (confirmed)"	14	["customer-profile-svc", "rtp-svc", "sanctions-screening", "wire-svc", "loan-origination-svc", "payee-verification-svc"]
"fails (assumed)"	34	["auth-gateway", "entitlements-svc", "mobile-bff", "p2p-transfer-svc", "web-bff", "ach-origination-svc"]
"degrades only"	28	["acquiring-gateway", "aml-screening-svc", "atm-switch", "card-auth-svc", "fraud-scoring-svc", "kyc-svc"]

3 rows

What to notice: 71% of the "failing" list is assumption.

Step 5: What does that mean for customers?

How many journey attempts per hour are affected, now (03:00) and as the morning builds (07:00)?

```
MATCH (j:Journey)
WITH j,
  EXISTS {
    MATCH (j)-[:REQUIRES]->(s:Service)
    MATCH p = (s)-[:DEPENDS_ON*1..8]->(:Datastore {name: 'profile-cache'})
    WHERE all(r IN relationships(p) WHERE r.confirmed)
  } AS confirmed,
  EXISTS {
    MATCH (j)-[:REQUIRES]->(s:Service)
    MATCH p = (s)-[:DEPENDS_ON*1..8]->(:Datastore {name: 'profile-cache'})
    WHERE all(r IN relationships(p) WHERE r.hard)
  } AS assumed
UNWIND [{scenario: 'best case (proven only)', down: confirmed},
       {scenario: 'worst case (assume the unknowns)', down: assumed}] AS sc
WITH sc.scenario AS scenario, j, sc.down AS down
WHERE down
RETURN scenario, count(j) AS journeys_down,
       sum(j.per_hour_0300) AS attempts_per_hour_at_0300,
       sum(j.per_hour_0700) AS attempts_per_hour_at_0700,
       sum(j.per_hour_1200) AS attempts_per_hour_at_1200
ORDER BY journeys_down;
```

You should see a range, not a number.

scenario	journeys_down	attempts_per_hour_at_0300	attempts_per_hour_at_0700	attempts_per_hour_at_1200
"best case (proven only)"	4	473	6380	16480
"worst case (assume the unknowns)"	26	32040	319720	474060

2 rows

What to notice: at 03:00 the answer is between **473** and **32,040** attempts an hour. **A factor of about 68, because of what nobody has checked.** And by 07:00 it's 10 times higher.

Part 3: Why can't we tell? (about 10 minutes)

03:25. Something is making the answer a range. Find out what.

Step 6: The one link worth checking

These are the dependencies on the path to the cache that **nobody ever classified**, ranked by how much sits above each.

```
MATCH (a:Component)-[:DEPENDS_ON]->(b:Component)
WHERE e.active AND e.critical IS NULL
AND EXISTS {
    MATCH p = (b)-[:DEPENDS_ON*0..8]->(:Datastore {name: 'profile-cache'})
    WHERE all(r IN relationships(p) WHERE r.hard)
}
CALL {
    WITH a
    OPTIONAL MATCH p = (u:Service)-[:DEPENDS_ON*1..8]->(a)
    WHERE all(r IN relationships(p) WHERE r.hard)
    RETURN count(DISTINCT u) AS above
}
CALL {
    WITH a
    MATCH (j:Journey)-[:REQUIRES]->(s:Service)
    WHERE s = a OR EXISTS {
        MATCH p = (s)-[:DEPENDS_ON*1..8]->(a)
        WHERE all(r IN relationships(p) WHERE r.hard)
    }
    RETURN count(DISTINCT j) AS journeys
}
RETURN a.name + ' -> ' + b.name AS unclassified_dependency, e.source AS seen_via,
    toString(e.first_seen) AS first_seen, above + 1 AS services_at_stake, journeys AS journeys_at_stake
ORDER BY journeys_at_stake DESC, services_at_stake DESC, unclassified_dependency
LIMIT 6;
```

You should see 6 rows. The top one is `entitlements-svc -> customer-profile-svc`, with **33 services** and **26 journeys** above it. The next has 8 services and 5 journeys.

unclassified_dependency	seen_via	first_seen	services_at_stake	journeys_at_stake
"entitlements-svc -> customer-profile-svc"	"observed"	"2025-06-03"	33	26
"legacy-profile-lookup -> customer-profile-svc"	"observed"	"2024-11-02"	8	5
"payee-directory-svc -> legacy-profile-lookup"	"observed"	"2023-08-31"	4	4
"sanctions-screening -> legacy-profile-lookup"	"observed"	"2024-08-16"	3	1
"agent-desktop-svc -> customer-profile-svc"	"observed"	"2023-11-10"	2	1
"preference-sync-svc -> customer-profile-svc"	"observed"	"2023-09-21"	1	0

6 rows

What to notice: `seen_via` is `observed`. This link was seen in traces. **It was never declared in the catalog.**

Step 7: Where did it come from?

```
MATCH (a:Component {name: 'entitlements-svc'})-[:DEPENDS_ON]->(b:Component {name: 'customer-profile-svc'})
MATCH (ch:Change {id: e.introduced_by})-[:DEPLOYED_TO]->(:Service)<-[:OWNS]-(t:Team)
RETURN a.name + ' -> ' + b.name AS dependency, e.source AS seen_via,
    toString(e.first_seen) AS first_seen, ch.id AS introduced_by, ch.what AS change,
    t.name AS team, ch.risk_review AS had_risk_review,
    duration.inMonths(e.first_seen, date('2026-09-30')).months AS months_in_production;
```

You should see the change that introduced it: `CHG-1873`, on 2025-06-03, **15 months ago**, with **no risk review**.

dependency	seen_via	first_seen	introduced_by	change	team	had_risk_review	months_in_production
"entitlements-svc -> customer-profile-svc"	"observed"	"2025-06-03"	"CHG-1873"	"Resolve group membership via customer-profile-svc"	"Identity & Access"	FALSE	15

1 row

What to notice: nobody did anything wrong. A team resolved group membership through the profile service, which is a perfectly normal change. It just never made it into the catalog.

Step 8: The replay

How big was the blast radius the day *before* that change, compared with tonight? The same question, asked "as of" two dates.

Predict: how many journeys depended on the cache the day before?

```
MATCH (ch:Change {id: 'CHG-1873'})
WITH date(ch.deployed_at) AS changed
UNWIND [changed - duration('P1D'), date('2026-09-30')] AS asof
CALL {
  WITH asof
  MATCH (s:Service)
  WHERE EXISTS {
    MATCH p = (s)-[:DEPENDS_ON*1..8]->(:Datastore {name: 'profile-cache'})
    WHERE all(r IN relationships(p) WHERE r.hard AND r.first_seen <= asof)
  }
  RETURN count(s) AS services_failing
}
CALL {
  WITH asof
  MATCH (j:Journey)
  WHERE EXISTS {
    MATCH (j)-[:REQUIRES]->(s:Service)
    MATCH p = (s)-[:DEPENDS_ON*1..8]->(:Datastore {name: 'profile-cache'})
    WHERE all(r IN relationships(p) WHERE r.hard AND r.first_seen <= asof)
  }
  RETURN count(j) AS journeys_down, sum(j.customers_per_day) AS attempts_per_day
}
RETURN toString(asof) AS as_of, services_failing, journeys_down, attempts_per_day
ORDER BY as_of;
```

You should see the worst case grow from **9 journeys to 26**, and from 22 services to 48.

as_of	services_failing	journeys_down	attempts_per_day
"2025-06-02"	22	9	1352000
"2026-09-30"	48	26	6704000

2 rows

What to notice: one change, 15 months ago, took the worst case from **9 journeys to 26**: about **5 times** the daily attempts at risk.

Part 4: Your turn: classify the link (about 8 minutes)

03:40. So far you've only *read* the graph. Now change it, in your own database, and watch the answer move.

The link is `entitlements-svc -> customer-profile-svc`. Nobody knows whether it's hard or soft. You'll try both answers.

Do these steps in order. They write to *your* database only. At any point, Step 14 puts everything back.

Step 9: The scoreboard, before

```
MATCH (j:Journey)
WITH j,
  EXISTS {
    MATCH (j)-[:REQUIRES]->(s:Service)
    MATCH p = (s)-[:DEPENDS_ON*1..8]->(:Datastore {name: 'profile-cache'})
    WHERE all(r IN relationships(p) WHERE r.hard)
  } AS worst,
  EXISTS {
    MATCH (j)-[:REQUIRES]->(s:Service)
    MATCH p = (s)-[:DEPENDS_ON*1..8]->(:Datastore {name: 'profile-cache'})
    WHERE all(r IN relationships(p) WHERE r.confirmed)
  } AS proven
RETURN sum(CASE WHEN worst THEN 1 ELSE 0 END) AS journeys_down_worst_case,
       sum(CASE WHEN proven THEN 1 ELSE 0 END) AS journeys_down_proven;
```

journeys_down_worst_case	journeys_down_proven
26	4

1 row

Write these down: **26** journeys down in the worst case, and **4** proven.

Step 10: Decide the link is SOFT

Suppose `entitlements-svc` survives a slow profile service. This marks the link soft and recomputes the `hard` and `confirmed` flags.

```
MATCH (:Component {name: 'entitlements-svc'})-[e:DEPENDS_ON]->(:Component {name: 'customer-profile-svc'})
SET e.critical = false
WITH count(e) AS links_classified
MATCH ()-[x:DEPENDS_ON]->()
SET x.hard = x.active AND coalesce(x.critical, true),
    x.confirmed = x.active AND coalesce(x.critical, false)
RETURN links_classified, count(x) AS flags_refreshed;
```

Step 11: The scoreboard again

Predict: how many journeys are down in the worst case now?

```
MATCH (j:Journey)
WITH j,
  EXISTS {
    MATCH (j)-[:REQUIRES]->(s:Service)
    MATCH p = (s)-[:DEPENDS_ON*1..8]->(:Datastore {name: 'profile-cache'})
    WHERE all(r IN relationships(p) WHERE r.hard)
  } AS worst,
  EXISTS {
    MATCH (j)-[:REQUIRES]->(s:Service)
    MATCH p = (s)-[:DEPENDS_ON*1..8]->(:Datastore {name: 'profile-cache'})
    WHERE all(r IN relationships(p) WHERE r.confirmed)
  } AS proven
RETURN sum(CASE WHEN worst THEN 1 ELSE 0 END) AS journeys_down_worst_case,
       sum(CASE WHEN proven THEN 1 ELSE 0 END) AS journeys_down_proven;
```

journeys_down_worst_case	journeys_down_proven
9	4

1 row

What to notice: the worst case falls to **9 journeys**. Tonight looks exactly like the day before `CHG-1873`. (*Curious what a graph algorithm makes of that change? That's stretch challenge S12, if your database has Graph Data Science.*)

Step 12: Now decide it's HARD

The opposite: `entitlements-svc` fails when the profile service is slow.

```
MATCH (:Component {name: 'entitlements-svc'})-[e:DEPENDS_ON]->(:Component {name: 'customer-profile-svc'})
SET e.critical = true
WITH count(e) AS links_classified
MATCH ()-[x:DEPENDS_ON]->()
SET x.hard = x.active AND coalesce(x.critical, true),
    x.confirmed = x.active AND coalesce(x.critical, false)
RETURN links_classified, count(x) AS flags_refreshed;
```

Step 13: The scoreboard once more

Predict: how many journeys are now *proven* down?

```
MATCH (j:Journey)
WITH j,
    EXISTS {
        MATCH (j)-[:REQUIRES]->(s:Service)
        MATCH p = (s)-[:DEPENDS_ON*1..8]->(:Datastore {name: 'profile-cache'})
        WHERE all(r IN relationships(p) WHERE r.hard)
    } AS worst,
    EXISTS {
        MATCH (j)-[:REQUIRES]->(s:Service)
        MATCH p = (s)-[:DEPENDS_ON*1..8]->(:Datastore {name: 'profile-cache'})
        WHERE all(r IN relationships(p) WHERE r.confirmed)
    } AS proven
RETURN sum(CASE WHEN worst THEN 1 ELSE 0 END) AS journeys_down_worst_case,
    sum(CASE WHEN proven THEN 1 ELSE 0 END) AS journeys_down_proven;
```

journeys_down_worst_case	journeys_down_proven
26	26

1 row

What to notice: all 26 are now proven. What was doubt is now fact. *One link, two answers, and nobody on the call knows which is true.*

Step 14: Put it back

```
MATCH (:Component {name: 'entitlements-svc'})-[e:DEPENDS_ON]->(:Component {name: 'customer-profile-svc'})
REMOVE e.critical
WITH count(e) AS links_reset
MATCH ()-[x:DEPENDS_ON]->()
SET x.hard = x.active AND coalesce(x.critical, true),
    x.confirmed = x.active AND coalesce(x.critical, false)
RETURN links_reset, count(x) AS flags_refreshed;
```

Optional check: run the scoreboard one more time. It should match Step 9 exactly.

```
MATCH (j:Journey)
WITH j,
    EXISTS {
        MATCH (j)-[:REQUIRES]->(s:Service)
        MATCH p = (s)-[:DEPENDS_ON*1..8]->(:Datastore {name: 'profile-cache'})
        WHERE all(r IN relationships(p) WHERE r.hard)
    } AS worst,
    EXISTS {
        MATCH (j)-[:REQUIRES]->(s:Service)
        MATCH p = (s)-[:DEPENDS_ON*1..8]->(:Datastore {name: 'profile-cache'})
        WHERE all(r IN relationships(p) WHERE r.confirmed)
    } AS proven
RETURN sum(CASE WHEN worst THEN 1 ELSE 0 END) AS journeys_down_worst_case,
    sum(CASE WHEN proven THEN 1 ELSE 0 END) AS journeys_down_proven;
```

If anything looks off: run Step 14 again. If it still looks wrong, ask a helper; they can restore your database in seconds.

Part 5: What do we do? (about 5 minutes)

03:52. You have an answer with a range and a cause. Now act on it.

Step 15: What changed?

Every change in the 24 hours before the page, ranked by how close it is to the cache.

```
MATCH (ch:Change)-[:DEPLOYED_TO]->(s:Service)
WHERE ch.deployed_at >= datetime('2026-09-30T03:07:00Z') - duration('PT24H')
AND ch.deployed_at <= datetime('2026-09-30T03:07:00Z')
OPTIONAL MATCH p = (s)-[:DEPENDS_ON*1..8]->(:Datastore {name: 'profile-cache'})
WHERE all(r IN relationships(p) WHERE r.hard)
WITH ch, s, min(length(p)) AS hops
RETURN ch.id AS change, s.name AS service, ch.what AS what,
       substring(toString(ch.deployed_at), 0, 16) AS deployed_at,
       duration.inSeconds(ch.deployed_at, datetime('2026-09-30T03:07:00Z')).hours AS hours_before_page,
       CASE WHEN hops IS NULL THEN 'no' ELSE 'yes' END AS in_radius, hops AS hops_from_cache,
       ch.risk_review AS had_risk_review
ORDER BY hops IS NULL, hops, ch.deployed_at DESC;
```

You should see 12 changes, 4 of them in the blast radius.

change	service	what	deployed_at	hours_before_page	in_radius	hops_from_cache	had_risk_review
"CHG-2301"	"customer-profile-svc"	"Config: profile cache TTL 24h to 1h"	"2026-09-29T22:15"	4	"yes"	1	FALSE
"CHG-2304"	"entitlements-svc"	"Dependency upgrade: http client 4.2"	"2026-09-29T19:40"	7	"yes"	2	TRUE
"CHG-2299"	"bill-pay-svc"	"Deploy v2.8"	"2026-09-29T14:00"	13	"yes"	3	TRUE
"CHG-2306"	"mobile-bff"	"Feature flag rollout: new login screen 25%"	"2026-09-29T17:00"	10	"yes"	4	TRUE
"CHG-2305"	"tracing-collector"	"Config: sampling rate 10% to 5%"	"2026-09-30T01:10"	1	"no"	NULL	TRUE
"CHG-2312"	"trade-finance-svc"	"Deploy v3.4.7"	"2026-09-30T01:04"	2	"no"	NULL	TRUE
"CHG-2311"	"consent-ledger"	"Deploy v8.3.9"	"2026-09-30T00:04"	3	"no"	NULL	TRUE
"CHG-2310"	"batch-scheduler"	"Dependency upgrade: metrics agent"	"2026-09-29T21:31"	5	"no"	NULL	TRUE
"CHG-2303"	"card-dispute-svc"	"Deploy v1.9"	"2026-09-29T20:00"	7	"no"	NULL	TRUE
"CHG-2309"	"log-pipeline"	"Config: log level change"	"2026-09-29T13:22"	13	"no"	NULL	TRUE
"CHG-2308"	"treasury-reporting-svc"	"Dependency upgrade: http client"	"2026-09-29T07:13"	19	"no"	NULL	TRUE
"CHG-2307"	"password-reset-svc"	"Feature flag rollout 25%"	"2026-09-29T03:37"	23	"no"	NULL	TRUE

12 rows

What to notice: the closest is CHG-2301, a cache **TTL cut from 24 hours to 1 hour**, deployed at 22:15 with no risk review. The changes outside the radius are correctly ruled out.

Step 16: What does routing cost?

Across all 211 incidents with a recorded root cause, what happens when the ticket goes to the team that owns the root cause, compared with anywhere else?

```
MATCH (i:Incident)-[:ROOT_CAUSE]->(c:Component)
OPTIONAL MATCH (o:Team)-[:OWNS]->(c)
MATCH (i)-[:ASSIGNED_TO]->(a:Team)
WITH i, CASE WHEN a = o THEN 'assigned to the owner' ELSE 'assigned elsewhere' END AS routing
RETURN routing, count(i) AS incidents,
       round(avg(i.duration_minutes)) AS avg_minutes, percentileCont(i.duration_minutes, 0.5) AS median_minutes,
       round(avg(i.reassignments) * 10) / 10 AS avg_reassignments
ORDER BY routing;
```

routing	incidents	avg_minutes	median_minutes	avg_reassignments
"assigned elsewhere"	62	169.0	161.5	2.1
"assigned to the owner"	149	50.0	48	0.3

2 rows

What to notice: tickets that went to the owner took **50 minutes** on average. Tickets routed elsewhere took **169 minutes**, bouncing between teams **2.1** times. *These incident figures are invented for the exercise, and the size of the gap is built into the data. The direction is the point.*

The decision

You have, at 03:55:

1. **Data Platform:** roll back `CHG-2301`. It's the closest change, and the one you can undo.
2. **Identity & Access:** answer one question: *does `entitlements-svc` survive a slow profile service?* It moves the worst case from 26 journeys to 9.
3. **Tell the other teams in the blast radius.**
4. **Monday:** classify the link, declare it, give the cache a runbook and an honest tier, and route incidents to the team that owns the cause.

Three things to take away:

1. **The dependency that matters isn't in the catalog.** It's in the traces, and in the graph.
2. **"We don't know" is measurable.** The graph tells you how much of the answer is assumption, and which single fact would change it most.
3. **Much of an incident's cost is routing, not fixing.**

Stretch challenges (if you finish early)

Try each one first; the query and result are under each. **Keep variable-length paths short** (`*1..8` or less); everyone's database lives on the same server. **S12 and S13 need the Graph Data Science (GDS) plugin**, which not every database has; each tells you what to do if yours doesn't.

S1: Which journeys are down?

Every journey, with its status: proven down, assumed down, or up.

Solution: see "S1: Which journeys are down?" in the Answers section at the back.

S2: Three scenarios side by side

Without editing anything: journeys down in the worst case, if the link from Step 6 is soft, and proven only.

Solution: see "S2: Three scenarios side by side" in the Answers section at the back.

S3: Who do we page?

Solution: see "S3: Who do we page?" in the Answers section at the back.

S4: Who can we not page?

Solution: see "S4: Who can we not page?" in the Answers section at the back.

S5: Which other datastores are declared lower than they should be?

Solution: see "S5: Which other datastores are declared lower than they should be?" in the Answers section at the back.

S6: The chain, link by link

Why does a journey depend on the cache? Show the path and the status of every link.

Solution: see "S6: The chain, link by link" in the Answers section at the back.

S7: Have we been here before?

Solution: see "S7: Have we been here before?" in the Answers section at the back.

S8: What does history say about the unclassified link?

Solution: see "S8: What does history say about the unclassified link?" in the Answers section at the back.

S9: Are we covered at 3 a.m.?

Solution: see "S9: Are we covered at 3 a.m.?" in the Answers section at the back.

S10: What do the numbers say?

PageRank over the hard dependencies, computed ahead of time, independent of anything declared.

Solution: see "S10: What do the numbers say?" in the Answers section at the back.

S11: Is there anything else in here?

This story was about one cache. But a graph this size can hold more than one problem. **Is there another hotspot hiding in it that this guide has never mentioned?** One person in the room will find it first.

Solution: see "S11: Is there anything else in here? (Hint 1: what is in the graph?)" in the Answers section at the back.

Solution: see "S11: Is there anything else in here? (Hint 2 and solution)" in the Answers section at the back.

S12: Run the algorithm yourself, then change the answer

Needs the Graph Data Science (GDS) plugin. If your database doesn't have it, this stops with an error such as *There is no procedure with the name gds.graph.drop registered*. Nothing was changed; skip S12 and S13. (If the error says you are *not allowed* to run it, tell a helper.)

S10 read a score that was worked out in advance. This runs **PageRank now**, over the hard dependencies as they are at this moment: it builds a temporary in-memory copy of them, runs the algorithm, and removes the copy. It writes nothing to your database. It shows the top five, plus the three components the story is about.

Solution: see "S12: Run the algorithm yourself, then change the answer" in the Answers section at the back.

S13: Does the org chart match the dependency map?

Needs GDS, like S12. *Louvain* finds groups of components that depend on each other more than on the rest of the estate, ignoring direction. For each group this shows how many **teams** own its members, which team owns the most, and how many belong to the team that owns the cache.

Solution: see "S13: Does the org chart match the dependency map?" in the Answers section at the back.

If something goes wrong

What you see	What to do
Every query returns no rows , or the readiness check shows 0 components	You're on the wrong database. Raise your hand; don't try to fix it
An error saying something " is not allowed " on database <code>neo4j</code> , or " Database not found " / access denied	Your login opened the wrong database. Raise your hand
Syntax error	Part of the query was probably missed when copying. Copy the whole block again, from the first line to the <code>;</code>
A query keeps spinning	Click the stop button on its frame. Lab queries finish in under a second; a slow one is usually an edited query with a long <code>*</code> path
Your numbers differ from the guide after Part 4	Run Step 14 again
Your data looks deleted or changed	Raise your hand. We can restore your database in seconds

After the session

- **Take it with you.** The repository (or zip) from your pre-event email has this guide, every query, and the graph. `graph/load.cypher` rebuilds the whole lab on your own Neo4j 5.26 or later; instructions are at the top of that file.
- **Learn more Cypher and graph data science** at [Neo4j GraphAcademy](#), which has free, self-paced courses.

Glossary

Term	Meaning
Blast radius	Everything that stops working, or gets worse, if one component fails
Journey	A thing customers do (Log in, Pay a bill). Each needs several services
Attempt	One customer trying one journey. Two journeys by one person are two attempts
Tier	How critical a component is. 1 = customer-facing critical, 3 = can wait until morning
Hard / soft / unclassified	A dependency the caller fails without / one it can work around / one nobody ever decided
Proven vs assumed	Down because every link is a known hard dependency / down only because an unclassified link is assumed hard
Hop	One step along a dependency
Hit rate	The share of cache lookups answered from the cache. When it falls, the database behind it takes the load
TTL	How long a cache entry lives before it expires
p99	The latency that 99% of requests beat. A measure of the slow tail
Runbook	Written steps for handling a known failure
Root cause	The component that actually failed, as opposed to those that merely showed symptoms
PageRank	A score for how much of a graph leans on each node, counting indirect reliance. Here, how much of the estate depends on a component, directly or through others
Community	A group of nodes more connected to each other than to the rest, found by an algorithm (Louvain). Here, components that depend on each other

Answers

Solutions to the stretch challenges. No peeking until you have tried.

S1: Which journeys are down?

```
MATCH (j:Journey)
WITH j,
  EXISTS {
    MATCH (j)-[:REQUIRES]->(s:Service)
    MATCH p = (s)-[:DEPENDS_ON*1..8]->(:Datastore {name: 'profile-cache'})
    WHERE all(r IN relationships(p) WHERE r.confirmed)
  } AS confirmed,
  EXISTS {
    MATCH (j)-[:REQUIRES]->(s:Service)
    MATCH p = (s)-[:DEPENDS_ON*1..8]->(:Datastore {name: 'profile-cache'})
    WHERE all(r IN relationships(p) WHERE r.hard)
  } AS assumed
RETURN j.name AS journey,
  CASE WHEN confirmed THEN 'DOWN (confirmed)' WHEN assumed THEN 'DOWN (assumed)' ELSE 'up' END AS status,
  j.per_hour_0300 AS per_hour_at_0300, j.per_hour_0700 AS per_hour_at_0700
ORDER BY CASE WHEN confirmed THEN 0 WHEN assumed THEN 1 ELSE 2 END, j.customers_per_day DESC;
```

journey	status	per_hour_at_0300	per_hour_at_0700
"Update contact details"	"DOWN (confirmed)"	425	4250
"Visit a branch teller"	"DOWN (confirmed)"	0	660
"Send a wire"	"DOWN (confirmed)"	45	1350
"Apply for a mortgage"	"DOWN (confirmed)"	3	120
"Log in"	"DOWN (assumed)"	12000	120000
"Check a balance"	"DOWN (assumed)"	8000	80000
"Pay a credit card"	"DOWN (assumed)"	2650	26500
"Transfer money"	"DOWN (assumed)"	2100	21000
"Pay a bill"	"DOWN (assumed)"	1240	13950
"Deposit a check"	"DOWN (assumed)"	660	8800
"Chat with support"	"DOWN (assumed)"	1260	8400
"Make a loan payment"	"DOWN (assumed)"	850	8500
"Add a payee"	"DOWN (assumed)"	600	6000
"Download a statement"	"DOWN (assumed)"	475	4750
"Set up autopay"	"DOWN (assumed)"	450	4500
"Freeze a card"	"DOWN (assumed)"	600	2400
"Open an account"	"DOWN (assumed)"	104	1560
"Redeem rewards"	"DOWN (assumed)"	220	2200
"Place a trade"	"DOWN (assumed)"	38	380
"Set a travel notice"	"DOWN (assumed)"	62	620
"Dispute a charge"	"DOWN (assumed)"	140	1400
"Message my advisor"	"DOWN (assumed)"	23	460
"View merchant settlements"	"DOWN (assumed)"	21	630
"Get an FX quote"	"DOWN (assumed)"	51	510
"Run payroll"	"DOWN (assumed)"	14	420
"Approve positive pay"	"DOWN (assumed)"	9	360
"Tap to pay"	"up"	22800	76000
"Withdraw cash at an ATM"	"up"	11700	35100
"Find a branch or ATM"	"up"	1040	7800
"Reset a password"	"up"	1120	7000

30 rows

22 are down only if the unclassified links turn out hard. *Tap to pay* survives either way: its link to the cache is soft, with a fallback.

S2: Three scenarios side by side

```
MATCH (j:Journey)
WITH j,
  EXISTS {
    MATCH (j)-[:REQUIRES]->(s:Service)
    MATCH p = (s)-[:DEPENDS_ON*1..8]->(:Datastore {name: 'profile-cache'})
    WHERE all(r IN relationships(p) WHERE r.hard)
  } AS worst,
  EXISTS {
    MATCH (j)-[:REQUIRES]->(s:Service)
    MATCH p = (s)-[:DEPENDS_ON*1..8]->(:Datastore {name: 'profile-cache'})
    WHERE all(r IN relationships(p) WHERE r.hard
      AND NOT (startNode(r).name = 'entitlements-svc' AND endNode(r).name = 'customer-profile-svc'))
  } AS if_soft,
  EXISTS {
    MATCH (j)-[:REQUIRES]->(s:Service)
    MATCH p = (s)-[:DEPENDS_ON*1..8]->(:Datastore {name: 'profile-cache'})
    WHERE all(r IN relationships(p) WHERE r.confirmed)
  } AS proven
UNWIND [{scenario: 'worst case (assume the unknowns)', down: worst},
  {scenario: 'if entitlements -> customer-profile is soft', down: if_soft},
  {scenario: 'proven only', down: proven}] AS sc
WITH sc.scenario AS scenario, j, sc.down AS down
WHERE down
ORDER BY j.name
RETURN scenario, count(j) AS journeys_down, collect(j.name) AS which
ORDER BY journeys_down DESC;
```

scenario	journeys_down	which
"worst case (assume the unknowns)"	26	["Add a payee", "Apply for a mortgage", "Approve positive pay", "Chat with support", "Check a balance", "Deposit a check", "Dispute a charge", "Download a statement", "Freeze a card", "Get an FX quote", "Log in", "Make a loan payment", "Message my advisor", "Open an account", "Pay a bill", "Pay a credit card", "Place a trade", "Redeem rewards", "Run payroll", "Send a wire", "Set a travel notice", "Set up autopay", "Transfer money", "Update contact details", "View merchant settlements", "Visit a branch teller"]
"if entitlements -> customer-profile is soft"	9	["Add a payee", "Apply for a mortgage", "Chat with support", "Pay a bill", "Send a wire", "Set up autopay", "Transfer money", "Update contact details", "Visit a branch teller"]
"proven only"	4	["Apply for a mortgage", "Send a wire", "Update contact details", "Visit a branch teller"]

3 rows

S3: Who do we page?

```
MATCH (c:Component)
WHERE c.name = 'profile-cache'
  OR EXISTS {
    MATCH p = (c)-[:DEPENDS_ON*1..8]->(:Datastore {name: 'profile-cache'})
    WHERE all(r IN relationships(p) WHERE r.hard)
  }
OPTIONAL MATCH (t:Team)-[:OWNS]->(c)
WITH coalesce(t.name, '(nobody)') AS team, t.status AS status, t.pager AS pager, c
ORDER BY c.name
RETURN team, status, pager, count(c) AS owns_in_radius, collect(c.name)[0..4] AS examples
ORDER BY owns_in_radius DESC, team;
```

team	status	pager	owns_in_radius	examples
"Payments Platform"	"active"	"payments-oncall"	9	["ach-origination-svc", "bill-pay-svc", "fx-quote-svc", "p2p-transfer-svc"]
"Digital Channels"	"active"	"digital-oncall"	7	["alerts-preferences-svc", "document-upload-svc", "mobile-bff", "open-account-svc"]
"Cards"	"active"	"cards-oncall"	5	["card-activation-svc", "card-controls-svc", "card-issuance-svc", "rewards-redemption-svc"]
"Lending"	"active"	"lending-oncall"	4	["collections-svc", "loan-origination-svc", "loan-servicing-svc", "payment-plan-svc"]
"Corporate Banking"	"active"	"corpbank-oncall"	3	["cash-mgmt-portal", "payroll-svc", "positive-pay-svc"]
"Customer Support Tech"	"active"	"support-oncall"	3	["agent-desktop-svc", "chat-gateway", "screen-pop-svc"]
"Data Platform"	"active"	"dataplat-oncall"	3	["customer-profile-svc", "customer-segments-svc", "profile-cache"]
"Identity & Access"	"active"	"iam-oncall"	3	["auth-gateway", "entitlements-svc", "sso-federation-svc"]
"Marketing Tech"	"active"	"martech-oncall"	3	["audience-builder", "campaign-svc", "offers-engine"]
"Wealth"	"active"	"wealth-oncall"	3	["advisor-portal-svc", "order-routing-svc", "trade-order-svc"]
"Branch & ATM Tech"	"active"	"branch-oncall"	1	["teller-app-svc"]
"Legacy Platform"	"disbanded"	NULL	1	["legacy-profile-lookup"]
"Merchant Services"	"active"	"merchant-oncall"	1	["merchant-portal-svc"]
"Messaging Platform"	"active"	"messaging-oncall"	1	["preference-sync-svc"]
"Shared Services"	"disbanded"	NULL	1	["auth-proxy"]
"Treasury Tech"	"active"	"treasury-oncall"	1	["fx-hedging-svc"]

16 rows

14 active teams own something in the radius, and so do 2 disbanded ones. Only two have a decision to make.

S4: Who can we not page?

```
MATCH (c:Component)
WHERE EXISTS {
    MATCH p = (c)-[:DEPENDS_ON*1..8]->(:Datastore {name: 'profile-cache'})
    WHERE all(r IN relationships(p) WHERE r.hard)
}
OPTIONAL MATCH (t:Team)-[:OWNS]->(c)
WITH c, t
WHERE t IS NULL OR t.status <> 'active'
RETURN c.name AS component, c.lifecycle AS lifecycle,
    coalesce(t.name, '(nobody)') AS last_owner, toString(t.disbanded_on) AS disbanded_on
ORDER BY component;
```

component	lifecycle	last_owner	disbanded_on
"auth-proxy"	"deprecated"	"Shared Services"	"2025-08-29"
"legacy-profile-lookup"	"deprecated"	"Legacy Platform"	"2025-03-31"

2 rows

`auth-proxy` and `legacy-profile-lookup` are in the blast radius and their owning teams were **disbanded**.

S5: Which other datastores are declared lower than they should be?

```
MATCH (d:Datastore)
CALL {
  WITH d
  MATCH (j:Journey)-[:REQUIRES]->(s:Service)
  WHERE EXISTS {
    MATCH p = (s)-[:DEPENDS_ON*1..8]->(d)
    WHERE all(r IN relationships(p) WHERE r.confirmed)
  }
  RETURN count(DISTINCT j) AS proven
}
CALL {
  WITH d
  MATCH (j:Journey)-[:REQUIRES]->(s:Service)
  WHERE EXISTS {
    MATCH p = (s)-[:DEPENDS_ON*1..8]->(d)
    WHERE all(r IN relationships(p) WHERE r.hard)
  }
  RETURN count(DISTINCT j) AS worst_case
}
WITH d, proven, worst_case,
CASE WHEN d.tier >= 2 AND proven >= 3 THEN 'MISMATCH'
      WHEN d.tier IS NULL AND proven >= 2 THEN 'UNRATED'
      WHEN d.tier >= 2 AND worst_case >= 3 THEN 'REVIEW'
      ELSE '' END AS flag
WHERE worst_case >= 3 OR flag <> ''
RETURN d.name AS datastore, d.kind AS kind, d.tier AS declared_tier,
       proven AS journeys_proven, worst_case AS journeys_worst_case, flag
ORDER BY CASE flag WHEN 'MISMATCH' THEN 0 WHEN 'UNRATED' THEN 1 WHEN 'REVIEW' THEN 2 ELSE 3 END,
         coalesce(d.tier, 0) DESC, proven DESC, worst_case DESC, datastore
LIMIT 12;
```

datastore	kind	declared_tier	journeys_proven	journeys_worst_case	flag
"profile-cache"	"redis"	3	4	26	"MISMATCH"
"risk-features-store"	"redis"	2	10	10	"MISMATCH"
"reference-data-db"	"postgres"	2	5	5	"MISMATCH"
"payee-db"	"postgres"	2	4	4	"MISMATCH"
"docs-store"	"object"	NULL	2	2	"UNRATED"
"feature-flags-redis"	"redis"	3	0	7	"REVIEW"
"legacy-profile-extract"	"oracle"	3	0	5	"REVIEW"
"vault-storage"	"vault"	1	27	27	""
"token-db"	"postgres"	1	26	26	""
"session-store"	"redis"	1	25	25	""
"ledger-db"	"oracle"	1	19	19	""
"limits-db"	"postgres"	1	7	7	""

12 rows

MISMATCH means declared tier 2 or 3 but **proven** to be needed by 3 or more journeys. **UNRATED** means never given a tier. **REVIEW** means only unclassified links put it there. There are **4 mismatches**; **profile-cache** heads the list because it was declared tier 3, the lowest, and sits under 26 journeys in the worst case.

S6: The chain, link by link

```
UNWIND [{journey: 'Send a wire', mode: 'confirmed'}, {journey: 'Log in', mode: 'hard'}] AS q
MATCH (j:Journey {name: q.journey}), (d:Datastore {name: 'profile-cache'})
MATCH p = shortestPath((j)-[:REQUIRES|DEPENDS_ON*1..9]->(d))
WHERE all(r IN relationships(p) WHERE type(r) = 'REQUIRES'
          OR (q.mode = 'confirmed' AND r.confirmed) OR (q.mode = 'hard' AND r.hard))
RETURN j.name AS journey, [n IN nodes(p) | n.name] AS chain,
       [r IN relationships(p) | CASE WHEN type(r) = 'REQUIRES' THEN 'requires'
                                     WHEN r.critical IS NULL THEN 'UNCLASSIFIED'
                                     WHEN r.critical THEN 'critical' ELSE 'soft' END] AS links;
```

journey	chain	links
"Send a wire"	["Send a wire", "wire-svc", "sanctions-screening", "customer-profile-svc", "profile-cache"]	["requires", "critical", "critical", "critical"]
"Log in"	["Log in", "auth-gateway", "entitlements-svc", "customer-profile-svc", "profile-cache"]	["requires", "critical", "UNCLASSIFIED", "critical"]

2 rows

Send a wire is proven end to end. *Log in* reaches the cache through exactly one **UNCLASSIFIED** link.

S7: Have we been here before?

```
MATCH (i:Incident)-[:ROOT_CAUSE]->(c:Component)
OPTIONAL MATCH (o:Team)-[:OWNS]->(c)
MATCH (i)-[:ASSIGNED_TO]->(a:Team)
WITH c, o, count(i) AS incidents, sum(i.duration_minutes) AS minutes,
      sum(i.customers_impacted) AS customers, collect(DISTINCT a.name) AS assigned_to,
      sum(CASE WHEN a = o THEN 1 ELSE 0 END) AS assigned_to_owner
RETURN c.name AS root_cause, coalesce(o.name, '(nobody)') AS owner, incidents,
       minutes AS total_minutes, customers AS customers_impacted,
       size(assigned_to) AS teams_assigned, assigned_to_owner
ORDER BY total_minutes DESC
LIMIT 5;
```

root_cause	owner	incidents	total_minutes	customers_impacted	teams_assigned	assigned_to_owner
"profile-cache"	"Data Platform"	6	1325	1824100	4	0
"event-bus"	"Data Platform"	5	393	90526	2	4
"payment-orchestrator"	"Payments Platform"	7	359	1399094	1	7
"tokenization-svc"	"Cards"	4	350	323955	3	2
"customer-profile-svc"	"Data Platform"	1	348	103257	1	0

5 rows

`profile-cache` caused **6** incidents and **1,325** minutes of impact (about 22 hours). They went to **4** different teams, and **none to the team that owns it**.

S8: What does history say about the unclassified link?

```
MATCH (i:Incident)-[:ROOT_CAUSE]->(:Datastore {name: 'profile-cache'})
MATCH (i)-[:ASSIGNED_TO]->(a:Team)
RETURN toString(date(i.started_at)) AS date, i.id AS incident, i.impact AS impact,
       i.duration_minutes AS minutes, a.name AS assigned_to, i.detail AS detail
ORDER BY date;
```

date	incident	impact	minutes	assigned_to	detail
"2025-07-08"	"INC-1041"	"degraded"	190	"Payments Platform"	"Cache evictions; hit rate fell to 44%"
"2025-10-21"	"INC-1058"	"degraded"	215	"Payments Platform"	"Cache evictions; hit rate fell to 41%"
"2026-01-14"	"INC-1077"	"degraded"	160	"Identity & Access"	"Cache evictions; hit rate fell to 38%; logins slow but succeeded"
"2026-03-30"	"INC-1093"	"outage"	305	"Lending"	"Cache evictions; hit rate fell to 21%"
"2026-06-17"	"INC-1120"	"degraded"	175	"Digital Channels"	"Cache evictions; hit rate fell to 36%"
"2026-08-11"	"INC-1135"	"outage"	280	"Digital Channels"	"Cache evictions; hit rate fell to 9%; entitlements-svc failed rather than slowed"

6 rows

In 4 of the 6 incidents the services above it *degraded*; in 2 it was an **outage**, when the hit rate fell to **21% and 9%**. Tonight it's **31% and falling**.

S9: Are we covered at 3 a.m.?

```
MATCH (c:Component)
WHERE c.name = 'profile-cache'
  OR EXISTS {
    MATCH p = (c)-[:DEPENDS_ON*1..8]->(:Datastore {name: 'profile-cache'})
    WHERE all(r IN relationships(p) WHERE r.hard)
  }
OPTIONAL MATCH (rb:Runbook)-[:COVERS]->(c)
WITH c, collect(rb) AS runbooks
WITH c, CASE WHEN size(runbooks) = 0 THEN 'no runbook'
             WHEN any(r IN runbooks WHERE NOT r.stale) THEN 'runbook (current)'
             ELSE 'runbook (stale)' END AS coverage
ORDER BY coalesce(c.tier, 9), c.name
RETURN coverage, count(*) AS components, collect(c.name)[0..6] AS most_critical_examples
ORDER BY coverage;
```

coverage	components	most_critical_examples
"no runbook"	35	["customer-profile-svc", "entitlements-svc", "ach-origination-svc", "agent-desktop-svc", "auth-proxy", "bill-pay-svc"]
"runbook (current)"	3	["auth-gateway", "mobile-bff", "rtp-svc"]
"runbook (stale)"	11	["p2p-transfer-svc", "sanctions-screening", "web-bff", "wire-svc", "cash-mgmt-portal", "payee-verification-svc"]

3 rows

S10: What do the numbers say?

```
MATCH (c:Component)
RETURN c.name AS component, labels(c)[1] AS kind, c.tier AS declared_tier,
       c.pagerank AS pagerank, c.cluster_id AS cluster
ORDER BY c.pagerank DESC, c.name
LIMIT 8;
```

component	kind	declared_tier	pagerank	cluster
"customer-profile-svc"	"Service"	1	2.4023	1
"ledger-svc"	"Service"	1	1.7644	2
"ledger-db"	"Datastore"	1	1.6497	2
"profile-db"	"Datastore"	1	1.4285	7
"entitlements-svc"	"Service"	1	1.4046	1
"profile-cache"	"Datastore"	3	1.171	1
"reference-data-svc"	"Service"	2	1.0193	4
"reference-data-db"	"Datastore"	2	1.0164	4

8 rows

customer-profile-svc ranks **first**, above ledger-svc; profile-cache, declared tier 3, is number 6.

S11: Is there anything else in here? (Hint 1: what is in the graph?)

Look at the labels. Does every kind of thing in this database appear in your guide?

```
MATCH (n)
UNWIND labels(n) AS label
RETURN label, count(*) AS nodes
ORDER BY nodes DESC, label;
```

label	nodes
"Change"	455
"Component"	232
"Incident"	228
"Service"	175
"Runbook"	71
"Datastore"	57
"Certificate"	34
"Journey"	30
"Team"	25
"Alert"	19
"Checkin"	1

11 rows

One of those labels has not come up once today.

S11: Is there anything else in here? (Hint 2 and solution)

Take a closer look at the label no one mentioned. Which of them is about to expire, and what depends on it?

```
MATCH (c:Certificate)
WITH c, duration.inDays(date('2026-09-30'), c.expires_on) AS days_left
WHERE days_left <= 60
CALL {
  WITH c
  OPTIONAL MATCH (i:Incident)-[:ROOT_CAUSE]->(c)
  RETURN toString(max(date(i.started_at))) AS last_expiry_outage, max(i.duration_minutes) AS last_outage_minutes
}
OPTIONAL MATCH (t:Team)-[:OWNS]->(c)
OPTIONAL MATCH (s:Service)-[:USES_CERT]->(c)
OPTIONAL MATCH (st:Team)-[:OWNS]->(s)
OPTIONAL MATCH (j:Journey)-[:REQUIRES]->(s)
RETURN c.name AS certificate, days_left AS expires_in_days,
       CASE c.auto_renew WHEN true THEN 'yes' WHEN false THEN 'NO' ELSE 'unknown' END AS renews_itself,
       coalesce(t.name, '(nobody)') + CASE WHEN t.status = 'disbanded' THEN ' (disbanded)' ELSE '' END AS owner,
       count(DISTINCT s) AS services, count(DISTINCT st) AS teams, count(DISTINCT j) AS journeys,
       toString(c.last_rotated) AS last_rotated, last_expiry_outage, last_outage_minutes
ORDER BY expires_in_days, certificate;
```

certificate	expires_in_days	renews_itself	owner	services	teams	journeys	last_rotated	last_expiry_outage	last_outage_minutes
"wildcard.internal.bank"	8	"NO"	"Shared Services (disbanded)"	38	16	27	"2025-10-10"	"2025-10-10"	240
"api-gateway.external"	12	"yes"	"Cloud Platform"	4	3	0	"2025-10-12"	NULL	NULL
"branch-wifi.radius"	14	"NO"	"Branch & ATM Tech"	2	1	0	"2025-10-14"	NULL	NULL
"sso.idp.signing"	19	"yes"	"Identity & Access"	3	1	24	"2025-10-19"	NULL	NULL
"acquiring.mtls"	27	"yes"	"Merchant Services"	3	1	1	"2025-10-27"	NULL	NULL

5 rows

wildcard.internal.bank expires in **8 days**. It does **not** renew itself. Its owner, **Shared Services**, was disbanded. It sits under **38 services** owned by **16 teams**, and **27 of the 30 customer journeys** touch one of them. It was last rotated on 2025-10-10, by hand, the morning it expired and took the estate down for 4 hours. The other certificates on the list are decoys: they expire soon but renew themselves, or matter little.

A different kind of hidden shared dependency, and an eight-day fuse instead of a 3 a.m. page.

S12: Run the algorithm yourself, then change the answer

```
CALL gds.graph.drop('lab_pagerank', false) YIELD graphName
WITH count(*) AS cleared
MATCH (s:Component)
OPTIONAL MATCH (s)-[r:DEPENDS_ON]->(t:Component)
WHERE r.hard
WITH gds.graph.project('lab_pagerank', s, t) AS g
CALL gds.pageRank.stream(g.graphName, {maxIterations: 50, dampingFactor: 0.85}) YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS c, score
ORDER BY score DESC, c.name
WITH collect({c: c, score: score}) AS ranked
CALL gds.graph.drop('lab_pagerank') YIELD graphName
UNWIND range(0, size(ranked) - 1) AS i
WITH ranked[i].c AS c, ranked[i].score AS score, i + 1 AS rank
WHERE rank <= 5 OR c.name IN ['profile-cache', 'entitlements-svc', 'customer-profile-svc']
RETURN rank, c.name AS component, c.tier AS declared_tier,
       round(score * 1000) / 1000.0 AS live_pagerank,
       round(c.pagerank * 1000) / 1000.0 AS precomputed
ORDER BY rank;
```

rank	component	declared_tier	live_pagerank	precomputed
1	"customer-profile-svc"	1	2.402	2.402
2	"ledger-svc"	1	1.764	1.764
3	"ledger-db"	1	1.65	1.65
4	"profile-db"	1	1.429	1.429
5	"entitlements-svc"	1	1.405	1.405
6	"profile-cache"	3	1.171	1.171

6 rows

live_pagerank and precomputed agree: the stored property is the same calculation, done earlier. profile-cache, declared tier 3, ranks **6**.

Now change the graph and run it again. Run **Step 10** (mark the link soft), then run this query once more:

rank	component	declared_tier	live_pagerank	precomputed
1	"ledger-svc"	1	1.764	1.764
2	"ledger-db"	1	1.65	1.65
3	"entitlements-svc"	1	1.405	1.405
4	"customer-profile-svc"	1	1.208	2.402
5	"reference-data-svc"	2	1.019	1.019
11	"profile-cache"	3	0.664	1.171

6 rows

customer-profile-svc falls from rank 1 to **4** (score 2.402 to 1.208) and profile-cache from rank 6 to **11** (1.171 to 0.664). The precomputed **column has not moved**: a stored score is a snapshot of the graph as it was when it was calculated. If you mark the link **hard** (Step 12) instead, nothing changes, because the worst case already assumed it was hard.

Put it back with Step 14 when you are done.

S13: Does the org chart match the dependency map?

```
CALL gds.graph.drop('lab_communities', false) YIELD graphName
WITH count(*) AS cleared
MATCH (s:Component)
OPTIONAL MATCH (s)-[r:DEPENDS_ON]->(t:Component)
WHERE r.hard
WITH gds.graph.project('lab_communities', s, t, {}, {undirectedRelationshipTypes: ['*']}) AS g
CALL gds.louvain.stream(g.graphName, {concurrency: 1}) YIELD nodeId, communityId
WITH communityId, collect(gds.util.asNode(nodeId)) AS members
WITH collect(members) AS communities
CALL gds.graph.drop('lab_communities') YIELD graphName
WITH communities, head([(t:Team)-[:OWNS]->(:Datastore {name: 'profile-cache'}) | t.name]) AS cache_owner
UNWIND communities AS members
WITH members, cache_owner,
    any(m IN members WHERE m.name = 'profile-cache') AS has_cache,
    [m IN members | head([(t:Team)-[:OWNS]->(m) | t.name])] AS owners
CALL {
    WITH owners
    UNWIND [o IN owners WHERE o IS NOT NULL] AS o
    WITH o, count(*) AS n
    ORDER BY n DESC, o
    RETURN count(*) AS teams, head(collect(o)) AS biggest_owner, head(collect(n)) AS biggest_owner_owns
}
WITH members, has_cache, teams, biggest_owner, biggest_owner_owns,
    size([o IN owners WHERE o = cache_owner]) AS owned_by_cache_owner
WHERE size(members) >= 5 OR has_cache
RETURN size(members) AS components, teams, biggest_owner, biggest_owner_owns,
    owned_by_cache_owner, has_cache AS has_profile_cache
ORDER BY components DESC, biggest_owner;
```

components	teams	biggest_owner	biggest_owner_owns	owned_by_cache_owner	has_profile_cache
36	9	"Payments Platform"	15	2	TRUE
32	8	"Core Banking"	14	0	FALSE
22	8	"Wealth"	9	2	FALSE
19	7	"Fraud & Risk"	7	1	FALSE
17	4	"Cards"	10	0	FALSE
17	4	"Cloud Platform"	8	0	FALSE
17	6	"Merchant Services"	5	2	FALSE
13	5	"Lending"	6	4	FALSE
13	5	"Messaging Platform"	7	0	FALSE
8	2	"Treasury Tech"	6	0	FALSE
6	3	"Marketing Tech"	3	2	FALSE

11 rows

The group that holds `profile-cache` has **36 components owned by 9 teams**. Payments Platform owns the most of them (15); Data Platform, which owns the cache, owns **2**. It is the finding from S7 seen from another side: the things that fail together are not inside one team, so a ticket for one of them lands in the wrong place.

The group sizes are repeatable here, but Louvain is a heuristic: a different version of GDS can split the smaller groups slightly differently. The group numbers in S10's `cluster` column were computed on another version for that reason.